VEB DASTURLAR XAVFSIZLIGI

A10. Veb kesh, CDN va HTTP protokoliga oid zamonaviy hujumlar

1-VARIANT — Kesh kaliti, shaxsiy javob va HTTP so‘rov chegarasini tekshirish

Mashg‘ulot maqsadi. Veb kesh bilan asosiy server qarorlari mos kelmaganda javoblar qanday noto‘g‘ri qayta tarqalishini tajribada ko‘rish; Cache Poisoning va Cache Deception sabablarini ajratish; himoyani sozlab, qonuniy keshlash saqlanganini tekshirish. HTTP Request Smuggling mexanizmi alohida baytlar modeli orqali tahlil qilinadi.

Kutiladigan natija. Talaba MISS, HIT va BYPASS holatlarini javob mazmuni va origin identifikatori bilan asoslaydi; kesh kalitiga kirmagan sarlavha hamda noto‘g‘ri statik yo‘l qoidasini aniqlaydi; shaxsiy javobni umumiy keshdan chiqaradi. CL va TE talqinlarida xabar chegarasi nega farqlanishini hisoblaydi.

Ko‘rsatkich	Mashg‘ulot sharti
Fan / semestr	Veb dasturlar xavfsizligi / faqat 5-semestr
Mashg‘ulot / shakl	A10; 1-variant; individual amaliy ish
Auditoriya	30 ta kompyuter; o‘zaro LAN; internet mavjud
Kompyuter	Windows 11; 8 yadroli CPU; RAM 32 GB; SSD 1 TB
Virtual muhit	VMware Workstation yoki VirtualBox; Ubuntu 24.04 LTS
VM resursi	2 vCPU, RAM 4 GB, disk 25 GB; NAT adapter
Vaqt / baho	80 daqiqa, oldindan tayyorlangan muhitda / 10 ball

Vositalar va tayyorgarlik

1. Python 3, curl va matn muharriri. Python standart kutubxonasi yetarli: pip, Docker va pullik vosita talab qilinmaydi.
2. Ilovadagi lab.py, framing.py hamda 1.1-bo‘limdagi settings.json darsdan oldin VMga joylanadi. OT va paketlarni o‘rnatish 80 daqiqaga kiritilmagan.

Ish chegarasi. Sinovlar faqat talabaning o‘z VMidagi 127.0.0.1 manzillarida bajariladi. Barcha ma’lumotlar sun’iy. CDN akkaunti yoki tashqi sayt kerak emas; model CDNning bitta kesh tuguni vazifasini soddalashtirib ko‘rsatadi.

Arxitektura va qisqa nazariya

lab.py bir jarayonda ikkita HTTP xizmatni ishga tushiradi. Edge — mijoz so‘rovini qabul qiluvchi kesh; origin — javobni hosil qiluvchi asosiy server. MISS yoki BYPASSda edge originga HTTP orqali murojaat qiladi; HITda saqlangan javobni qaytaradi.

Qism	Manzil	Vazifa
curl mijoz	VMning 2-terminali	Mustaqil so‘rovlar va sun’iy Cookie
Edge kesh	http://127.0.0.1:8080	Kesh kaliti, saqlash va qayta berish
Origin	http://127.0.0.1:8010	Ochiq kontent va sun’iy akkaunt javobi
Framing modeli	framing.py; tarmoqsiz	Baytlar ichida xabar chegarasini hisoblash

Mazkur kesh kaliti: (GET, Host, to‘liq yo‘l va query). X-Lab-Banner, Cookie va Authorization kalitga kiritilmagan. Javobga ta’sir qiluvchi kirish qiymati kalitda hisobga olinmasa, turli so‘rovlar bir xil javobga bog‘lanishi mumkin. [1]

Hodisa	Asosiy farq	Bu variantdagi dalil
Cache Poisoning	O‘zgartirilgan javob umumiy keshga tushadi	Sarlavhasiz so‘rovda A10-MARKER
Cache Deception	Shaxsiy sahifa statik resurs deb keshlanadi	Cookiesiz so‘rovda DEMO-ALICE
Request Smuggling	Ikki parser xabar tugashini turlicha belgilaydi	CL va TE uchun turli bayt chegarasi

Kesh siyosatidagi muhim tushunchalar

Cache-Control: no-store javobni saqlamaslikni bildiradi; private umumiy keshga saqlashni taqiqlaydi. no-cache esa saqlashni mutlaq taqiqlamaydi: qayta ishlatishdan oldin tasdiqlash talab etadi. max-age yangilik muddatini, s-maxage umumiy kesh uchun muddatni belgilaydi. Vary javob variantlarini tanlashda tegishli so‘rov sarlavhalarini hisobga olishga xizmat qiladi. [4]

Model chegarasi. Bu dastur RFC 9111ning to‘liq kesh implementatsiyasi emas: Vary, ETag, qayta tasdiqlash, ko‘p CDN tuguni va HTTP/2 bajarilmagan. Asosiy HTTP xizmatlari GET sinovlari uchun; request smuggling ular orqali bajarilmaydi. HTTP/1.1 talqini alohida framing.pyda o‘rganiladi.

1.1. Virtual muhit va fayllarni tayyorlash

1-qadam. Tayyor VM bo‘lmasa, VMware yoki VirtualBoxda Ubuntu 24.04 LTS ISO bilan yangi VM yarating: 2 vCPU, 4 GB RAM, 25 GB disk, NAT adapter. OTni o‘rnating va oddiy foydalanuvchi bilan kiring. Quyidagi buyruqlar Ubuntu Bash terminali uchun; Windows PowerShellda bajarilmaydi.

```
sudo apt update
sudo apt install -y python3 curl nano
mkdir -p ~/a10-lab
cd ~/a10-lab
```

2-qadam. 1-ilovadagi A, B va C qismlarni ketma-ket bitta lab.py fayliga, 2-ilovani framing.py fayliga saqlang. Sarlavhalarni kodga qo‘shmang. UTF-8, oddiy qo‘shtirnoq va Python surilishlarini saqlang. settings.json quyidagi boshlang‘ich holatda bo‘lsin:

```
cat > settings.json <<'EOF'
{
  "trust_banner": true,
  "loose_routes": true,
  "safe_cache": false
}
EOF
cp settings.json settings.before.json
python3 -m json.tool settings.json
python3 -m py_compile lab.py framing.py
python3 --version
curl --version
python3 -u lab.py 2>&1 | tee lab-before.log
```

1-terminal ochiq qoladi. A10: origin=8010 edge=8080 va config qiymatlari chiqadi. Endi 2-terminal oching:

```
cd ~/a10-lab
E=http://127.0.0.1:8080
O=http://127.0.0.1:8010
ss -ltn | grep -E ":(8010|8080)[[:space:]]"
curl --noproxy "*" -i "$O/public"
```

Ikki tinglovchi 127.0.0.1da bo‘lsin. Origin javobi 200, tanasi PUBLIC banner=NORMAL. X-Origin-Id — origin har bir GETni qayta ishlaganda oshiradigan raqam; u foydalanuvchi identifikatori emas.

Dalil D1. VM parametrlari, Python/curl versiyalari, faol sozlamalar, ikki lokal port va origin javobini saqlang. Yangi terminalda E va O o‘zgaruvchilarini qayta belgilang. Brauzer o‘rniga curl ishlatilishi uning alohida keshini tajribaga aralashtirmaydi.

1.2. Kesh kaliti va Cache Poisoning

3-topshiriq. Avval oddiy keshlashni kuzating. Bir xil URLni 300 soniya ichida ikki marta chaqiring:

```
curl --noproxy "*" -i "$E/public?case=base1"
curl --noproxy "*" -i "$E/public?case=base1"
```

Birinchi javob X-Lab-Cache: MISS, ikkinchisi HIT. Ikkalasida X-Origin-Id va tana bir xil. Age dastlab 0 bo‘lishi mumkin: so‘rovlar bir soniya ichida bajarilgan. X-Lab-Cache ushbu laboratoriyaning maxsus sarlavhasi, universal CDN standarti emas.

4-topshiriq. Endi hali ishlatilmagan case=p1 kalitiga markerli so‘rov yuboring. Keyingi so‘rovda marker sarlavhasini yubormang:

```
curl --noproxy "*" -i -H "X-Lab-Banner: A10-MARKER" \
  "$E/public?case=p1"
curl --noproxy "*" -i "$E/public?case=p1"
curl --noproxy "*" -i "$E/public?case=clean1"
```

So‘rov	Kesh holati	Javob tanasi
case=p1, marker sarlavhasi bilan	MISS	PUBLIC banner=A10-MARKER
case=p1, sarlavhasiz	HIT	PUBLIC banner=A10-MARKER
case=clean1, sarlavhasiz	MISS	PUBLIC banner=NORMAL

Sababni kod bilan bog‘lang

Edge trust_banner=true bo‘lgani uchun X-Lab-Bannerni originga uzatadi. Origin uni javob matniga qo‘shadi. Kesh kaliti esa bu sarlavhani hisobga olmaydi. Shuning uchun keyingi oddiy so‘rov oldingi o‘zgartirilgan javobni oladi. Faqat birinchi javobda marker ko‘rinishi kesh orqali tarqalishning dalili bo‘la olmaydi. [1]

case query qiymati bu modelda kalitga kiradi; clean1 bilan alohida toza nusxa hosil bo‘ladi. Haqiqiy CDNda query siyosati boshqacha bo‘lishi mumkin. Har sinov juftligida URL aynan bir xil, yangi tajribada esa case qiymati yangi bo‘lsin.

Dalil D2. Uch so‘rov, MISS/HIT, X-Origin-Id va tana. A10-MARKER oddiy matn: bu yerda JavaScript yoki XSS bajarilishi ko‘rsatilmaydi. Sinov 5 daqiqadan oshsa, yangi case=p2 bilan markerli so‘rovdan qayta boshlang.

1.3. Cache Deception: shaxsiy javobning tarqalishi

5-topshiriq. Origin /account yo‘lini shaxsiy sahifa deb biladi. Zaif marshrutlash /account/ bilan boshlanuvchi boshqa yo‘llarni ham shu sahifaga bog‘laydi. Edge esa .css bilan tugagan yo‘lni statik resurs deb saqlaydi. Shu talqinlar farqi kuzatiladi. [2]

```
curl --noproxy "*" -i -H "Cookie: lab_user=alice" \
  "$E/account/report.css?case=d1"
curl --noproxy "*" -i "$E/account/report.css?case=d1"
curl --noproxy "*" -i "$O/account/report.css?case=d1"
```

Holat	Status / kesh	Kutiladigan mazmun
Edge; Alice belgisi bilan	200 / MISS	PRIVATE user=alice code=DEMO-ALICE
Edge; Cookie yuborilmagan	200 / HIT	Aynan shu Alice ma’lumoti
Origin; Cookie yuborilmagan	401 / kesh yo‘q	LOGIN REQUIRED

Birinchi javobda Cache-Control: private, no-store bo‘lsa ham, safe_cache=false holati uni ataylab e’tiborsiz qoldiradi. Content-Type text/plain bo‘lishiga qaramay, .css qo‘shimchasi kesh qarorini belgilaydi. Faqat sarlavhani originda berish noto‘g‘ri edge qoidasini tuzatmaganini ko‘rsating.

Nazorat so‘rovlari

```
curl --noproxy "*" -i -H "Cookie: lab_user=bob" "$E/account"
curl --noproxy "*" -i "$E/account"
```

Bob uchun 200 va DEMO-BOB; cookiesiz so‘rov uchun 401. Ikkalasida BYPASS: /account .css bilan tugamaydi. Bu oddiy yo‘lda va chalg‘ituvchi qo‘shimchali yo‘lda kesh qarori farqlanganini ko‘rsatadi.

Dalil D3. Alice bilan olingan MISS, undan keyingi cookiesiz HIT, bir xil origin ID hamda bevosita originning 401 javobini birga saqlang. Ikkinchi so‘rovda -H Cookie bo‘lmasin; curl bu buyruqlarda cookie jar ishlatmaydi.

Sun’iy akkaunt chegarasi. lab_user=alice yoki bob faqat o‘quv shaxsini tanlaydi, xavfsiz autentifikatsiya emas. Haqiqiy parol, sessiya yoki shaxsiy ma’lumot kiritilmaydi. Real Cache Deceptionda foydalanuvchi autentifikatsiyalangan holda chalg‘ituvchi URLni ochishi mumkin; laboratoriyada shu holat qo‘lda modellashtiriladi.

1.4. Himoyani sozlash va qayta sinash

6-topshiriq. D2 va D3ni saqlagach, 1-terminalda Ctrl+C bosing. settings.jsonni quyidagicha o‘zgartiring. Qayta ishga tushirish xotiradagi eski keshni ham tozalaydi:

```
cd ~/a10-lab
cat > settings.json <<'EOF'
{
  "trust_banner": false,
  "loose_routes": false,
  "safe_cache": true
}
EOF
python3 -m json.tool settings.json
diff -u settings.before.json settings.json
python3 -u lab.py 2>&1 | tee lab-after.log
```

diff o‘zgarish topgani uchun chiqish kodi 1 bo‘lishi tabiiy. Faol config yangi qiymatlarni ko‘rsatsin. 2-terminalda oldingi ifloslangan URLlar bilan tekshiring:

```
curl --noproxy "*" -i -H "X-Lab-Banner: A10-MARKER" \
  "$E/public?case=p1"
curl --noproxy "*" -i "$E/public?case=p1"
curl --noproxy "*" -i -H "Cookie: lab_user=alice" \
  "$E/account/report.css?case=d1"
curl --noproxy "*" -i "$E/account/report.css?case=d1"
```

Poisoning juftligida MISS → HIT saqlanadi, lekin ikkala tana NORMAL. Deception juftligida 404 / BYPASS; Alice ma’lumoti yo‘q. Eski nusxani tozalash va yangi siyosatni yoqish ikkita alohida zarurat: faqat tozalash zaiflikni bartaraf etmaydi.

O‘zgartirish	Ishlash sababi
trust_banner=false	Edge mijoz bergan xizmat sarlavhasini originga uzatmaydi
loose_routes=false	Origin faqat aniq /account yo‘lini qabul qiladi
safe_cache=true	Faqat ruxsatli ochiq yo‘llar; Cookie/Authorizationda BYPASS; 200 va aniq PUBLIC siyosati kerak

Dalil D4. Sozlamalar farqi, yangi ishga tushish yozuvi va takroriy so‘rov natijalari. safe_cache ushbu sodda modelning ruxsat ro‘yxati: Cache-Control qiymatini PUBLIC satri bilan aynan solishtiradi; u umumiy HTTP sarlavha parseri emas.

1.5. Qonuniy xizmat va CDN siyosatini tekshirish

7-topshiriq. Tuzatish ochiq resursni keshlashni buzmaganini va akkaunt javobi ajratilganini tasdiqlang:

```
curl --noproxy "*" -i "$E/assets/app.css"
curl --noproxy "*" -i "$E/assets/app.css"
curl --noproxy "*" -i -H "Cookie: lab_user=alice" "$E/account"
curl --noproxy "*" -i "$E/account"
curl --noproxy "*" -i -H "Authorization: Bearer DEMO" \
  "$E/public?case=auth1"
```

CSS: 200, text/css, MISS → HIT, bir xil origin ID. /account: Alice bilan 200 / BYPASS, cookiesiz 401 / BYPASS. Authorizationli ochiq so‘rov 200 / BYPASS. DEMO token tekshirilmaydi: ushbu so‘rov faqat keshni chetlab o‘tish qoidasini sinaydi.

CDN uchun siyosat kartasi

Laboratoriya qarorlarini quyidagi jadvalga ko‘chiring. Bu CDN konsoliga tayyor import fayli emas; tanlangan provayderda qoidalar tartibi, origin sarlavhalari va kesh kaliti alohida tekshiriladi. Jadvalning dalil ustunini o‘z natijangiz bilan to‘ldiring.

Resurs / holat	Talab etiladigan qaror	Dalil
/assets/app.css, oddiy GET	Ochiq 200ni keshlash; 300 soniya	_____
/account va shaxsiy javob	Umumiy keshni chetlab o‘tish	_____
/account/report.css	Aniq marshrut: 404; keshlamaslik	_____
Cookie yoki Authorization	Bu laboratoriya siyosatida BYPASS	_____
X-Lab-Banner	Mijozdan kelgan qiymatni uzatmaslik	_____

CDN bir nechta edge tuguniga ega bo‘lsa, bitta tugundagi HIT yoki tozalash boshqalar holatini isbotlamaydi. Tuzatishda qoidani tarqatish, zararlangan nusxalarni purge qilish va qayta tekshirish rejalashtiriladi. Bu ishda faqat bitta lokal tugun bor; restart uning tozalash amali vazifasini bajaradi.

Dalil D5. CSS qayta keshlanishi, shaxsiy yo‘lning 200/401 BYPASS juftligi va siyosat kartasi. Barcha javoblar BYPASS bo‘lib qolishi topshiriqni to‘liq bajarish hisoblanmaydi: foydali ochiq keshlash saqlanishi kerak.

Qo‘shimcha tajriba — vaqt bo‘lsa

Yangi konfiguratsiyaning nusxasini saqlab, himoya belgilarini bittadan o‘zgartiring va har safar restart qiling. Qaysi himoya poisoningni, qaysi biri deceptionni cheklayotganini yozing. Yakunda uchala qiymatni himoyalangan holatga qaytaring. Bu tajriba asosiy 80 daqiqaga kiritilmagan.

1.6. HTTP Request Smuggling: bayt chegarasi

8-topshiriq. framing.py tarmoqqa ulanmaydi. U bitta tayyor HTTP/1.1 bayt ketma-ketligini ikki noto‘g‘ri kelishilgan parser nuqtayi nazaridan tahlil qiladi. CL.TE holatida old qatlam Content-Lengthga, keyingi qatlam Transfer-Encodingga tayansa, birinchi xabar turli joyda tugaydi. [3]

```
cd ~/a10-lab
python3 framing.py | tee framing-result.txt
```

Model tanasi 44 bayt: 0\r\n\r\n yakunlovchi chunk belgisi 5 bayt, GET /probe ... esa 39 bayt. Python b-satrlari CRLFni haqiqiy baytga aylantiradi. Ekrandagi \r va \n yozuvlarini bittadan ko‘rinadigan belgi deb sanamang.

Talqin	Chegara / qoldiq	Mazmuni
CL	end=133; qoldiq bo‘sh	Barcha 44 bayt POST tanasi hisoblanadi
TE	end=94; qoldiq GET /probe ...	Chunked tana 5 baytda tugagan
strict	REJECT ambiguous framing	CL+TE birga bo‘lgani uchun rad etadi
CL-only	Qabul; qoldiq bo‘sh	Butun tana CL bilan bir ma’noli
TE-only	Qabul; qoldiq GET /probe ...	Birinchi xabar tugagan; keyingi xabar baytlari qolgan

Hisob: sarlavhalar bloki 89 bayt; CL bo‘yicha 89+44=133, TE bo‘yicha 89+5=94. Farq 39 bayt. TE-only nazoratidagi qoldiqning o‘zi zaiflik emas: ikki qatlam ham shu chegarani tanlasa, keyingi xabar bir xil talqin qilinadi. Zaiflik uchun parserlar orasidagi nomuvofiqlik muhim.

Rad etish siyosati va zamonaviy kontekst

RFC 9112da TE CLdan ustun; ikkisi birga kelishi xato sifatida ko‘rilishi tavsiya qilinadi. Uzatishni tanlagan vositachi CLni olib tashlab, TEga muvofiq ishlashi shart. Bu model qat’iy rad etishni tanlaydi. Real shlyuzda noaniq xabarni 400 bilan rad etish va ulanishni yopish sinovdan o‘tkaziladi; bu skript 400 javobini yubormaydi. [5]

TE.CL teskari talqin farqidir; TE.TEda TE sarlavhasini tanishdagi nomuvofiqlik rol o‘ynaydi. HTTP/2dan HTTP/1.1ga o‘tish ham qayta formatlash chegarasini yaratadi. Bu variant ularni jonli bajarmaydi; HTTPS yoki mijoz tomonda HTTP/2 ishlashi barcha ichki qatlamlar mosligini o‘zi isbotlamaydi. [3]

Dalil D6. Dastur chiqishi, 89+44 va 89+5 hisobi, CL-only/TE-only nazoratlari. Parser faqat berilgan bo‘sh chunked namunasini tushunadi; to‘liq HTTP parser emas. Natija haqiqiy Apache, Nginx yoki CDNda smuggling borligini isbotlamaydi.

1.7. Vaqt, hisobot va baholash

Bosqich	Vaqt	Natija
Muhit va boshlang‘ich tekshiruv	8 daqiqa	D1
Oddiy kesh va poisoning	14 daqiqa	D2
Deception va nazorat so‘rovlari	13 daqiqa	D3
Tuzatish va foydali keshlash	18 daqiqa	D4–D5
HTTP chegaralarini tahlil qilish	17 daqiqa	D6
Hisobot va qisqa himoya	10 daqiqa	Yakuniy xulosa
Jami	80 daqiqa	Tayyorgarlik alohida

Hisobot tarkibi

3. Titul: fan, 5-semestr, A10, 1-variant, F.I.Sh., guruh va sana. VM hamda dastur versiyalari; edge–origin arxitekturasi.
4. D1–D6 dalillari: har birida buyruq, status, tegishli sarlavha va tana ko‘rinsin. Qayta ishlatilgan javobni X-Origin-Id bilan asoslang.
5. settings.before.json va yakuniy settings.json farqi, CDN siyosat kartasi, framing hisobi; 5–7 gap xulosa. Kutish bilan amaldagi natija farqlansa, sababini yozing.

Baholash mezonlari

Mezon	Ball
Muhit, portlar va keshning oddiy MISS/HIT holati	1
Poisoning: sarlavhasiz qayta tarqalish va sabab	2
Deception: cookiesiz HIT va origin 401 bilan qiyos	2
Tuzatish, shaxsiy BYPASS va ochiq CSS keshi	2
CL/TE hisobi, strict rad etish va nazorat talqini	2
Hisobot, CDN kartasi va xulosa	1
Jami	10

Qabul sharti. Faqat skrinshotdagi HIT yoki 200 yetarli emas. Poisoningda marker yuborilmagan so‘rov, deceptionda Cookie yuborilmagan so‘rov, himoyada esa ikkala salbiy holat yo‘qolishi va qonuniy keshlash davom etishi ko‘rsatiladi.

Yakunlash. 1-terminalda Ctrl+C bosing. lab-before.log, lab-after.log, framing-result.txt, sozlamalar va hisobotni saqlang. Zaif holatni qayta ko‘rish uchun settings.before.json nusxasi ishlatilishi mumkin; topshiriladigan yakuniy settings.json himoyalangan holatda qoladi.

Muammolar, nazorat savollari va manbalar

Belgi	Tekshirish / yechim
Address already in use	8080/8010da oldingi lab ishlayaptimi? Uning terminalida Ctrl+C bosing.
Connection refused	lab.py ishlayaptimi? Port va E/O qiymatlarini tekshiring.
Birinchi so‘rov HIT	Kalit avval ishlatilgan; yangi case qiymatini oling yoki restart qiling.
Ikkinchi so‘rov MISS	300 soniya o‘tganmi; URL, Host va query aynan bir xilmi?
Deception sinovi 401	Birinchi so‘rovda lab_user=alice Cookie bormi? Boshlang‘ich configmi?
Tuzatishdan keyin marker bor	Sozlama faqat startda o‘qiladi; jarayonni to‘xtatib qayta boshlang.
JSON / IndentationError	JSONda true/false kichik harfda; Python surilishlari va to‘liq kodni tekshiring.

Nazorat savollari

6. Poisoning va deceptionda keshga tushgan ma’lumotning kelib chiqishi nimasi bilan farq qiladi?
7. private, no-store ko‘rinib tursa ham nega D3da ma’lumot tarqaldi?
8. Keshni tozalash va qoidani tuzatish nega ikkita alohida amal?
9. CL va TE 39 baytga farq qilishi nimani anglatadi? TE-only qoldig‘i nega o‘zi zaiflik emas?
10. Bitta lokal HIT natijasi butun CDN haqidagi qaysi xulosalarni bermaydi?

Rasmiy manbalar

[1] PortSwigger — Web cache poisoning

[2] PortSwigger — Web cache deception

[3] PortSwigger — HTTP request smuggling

[4] RFC 9111 — HTTP Caching, 4.1 va 5.2

[5] RFC 9112 — HTTP/1.1, 6.3

Manbalar tekshirilgan sana: 07.09.2026. Mavzu taqdim etilgan fan dasturining A10 bandiga mos. Kodlar ushbu mashg‘ulot uchun yozilgan o‘quv modelidir.

O‘qituvchi uchun. Tarqatishdan oldin VMda uch faylni joylang va boshlang‘ich settings.jsonni tiklang. Tayyorlashda lokal modelning 22 ta HTTP tekshiruvi, TTL tugashi hamda framing chegaralari tekshirildi. Muayyan CDN yoki ishlab chiqarish serveriga oid zaiflik sinovi bajarilmagan.

1-ilova. lab.py — A qism

A, B va C qismlarni ketma-ket bitta ~/a10-lab/lab.py fayliga birlashtiring. Sarlavhani ko‘chirmang; surilishlarni saqlang.

```
import json
import time
import threading
from pathlib import Path
from http.client import HTTPConnection
from http.cookies import SimpleCookie
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import urlsplit

CFG = json.loads(Path(__file__).with_name('settings.json').read_text())
CACHE = {}
PUBLIC = 'public, max-age=300'
PRIVATE = 'private, no-store'


def reply(handler, status, headers, body):
    handler.send_response(status)
    for key, value in headers.items():
        handler.send_header(key, value)
    handler.send_header('Content-Length', str(len(body)))
    handler.end_headers()
    handler.wfile.write(body)


class Origin(BaseHTTPRequestHandler):
    count = 0

    def do_GET(self):
        Origin.count += 1
        path = urlsplit(self.path).path
        status, cc, mime = 200, PRIVATE, 'text/plain'
        if path == '/public':
            banner = self.headers.get('X-Lab-Banner', 'NORMAL')
            body, cc = 'PUBLIC banner=' + banner, PUBLIC
        elif path == '/assets/app.css':
            body, cc, mime = 'body {color: navy}', PUBLIC, 'text/css'
```

Origin javobni hosil qiladi. Uning hisoblagichi har bir GETda oshadi. PUBLIC va PRIVATE qiymatlari keyingi qismlardagi kesh qarorida ishlatiladi.

1-ilova. lab.py — B qism

A, B va C qismlarni ketma-ket bitta ~/a10-lab/lab.py fayliga birlashtiring. Sarlavhani ko‘chirmang; surilishlarni saqlang.

```
        elif path == '/account' or (
            CFG['loose_routes'] and path.startswith('/account/')
        ):
            cookies = SimpleCookie()
            cookies.load(self.headers.get('Cookie', ''))
            who = cookies['lab_user'].value if 'lab_user' in cookies else ''
            if who in ('alice', 'bob'):
                body = 'PRIVATE user=' + who + ' code=DEMO-' + who.upper()
            else:
                status, body = 401, 'LOGIN REQUIRED'
        else:
            status, body = 404, 'NOT FOUND'
        headers = {'Content-Type': mime, 'Cache-Control': cc,
                   'X-Origin-Id': str(Origin.count)}
        reply(self, status, headers, (body + '\n').encode())


class Edge(BaseHTTPRequestHandler):
    def do_GET(self):
        path = urlsplit(self.path).path
        key = ('GET', self.headers.get('Host', ''), self.path)
        if CFG['safe_cache']:
            eligible = path in ('/public', '/assets/app.css') and not (
                self.headers.get('Cookie') or self.headers.get('Authorization')
            )
        else:
            eligible = path == '/public' or path.endswith('.css')
        item = CACHE.get(key) if eligible else None
        if item and time.monotonic() - item[0] < 300:
            saved, status, headers, body = item
            headers = dict(headers, **{'X-Lab-Cache': 'HIT',
                           'Age': str(int(time.monotonic() - saved))})
            reply(self, status, headers, body)
            return
        forwarded = {}
        for name in ('Cookie', 'Authorization'):
```

CACHE faqat jarayon xotirasida saqlanadi. Kalitga self.path orqali query ham kiradi; HITda origin chaqirilmaydi. B qism C qismdagi davom bilan tugallanadi.

1-ilova. lab.py — C qism

A, B va C qismlarni ketma-ket bitta ~/a10-lab/lab.py fayliga birlashtiring. Sarlavhani ko‘chirmang; surilishlarni saqlang.

```
            if self.headers.get(name):
                forwarded[name] = self.headers[name]
        if CFG['trust_banner'] and self.headers.get('X-Lab-Banner'):
            forwarded['X-Lab-Banner'] = self.headers['X-Lab-Banner']
        conn = HTTPConnection('127.0.0.1', 8010, timeout=3)
        conn.request('GET', self.path, headers=forwarded)
        response = conn.getresponse()
        status, body = response.status, response.read()
        headers = {name: response.getheader(name) for name in
                   ('Content-Type', 'Cache-Control', 'X-Origin-Id')}
        conn.close()
        store = eligible and status == 200
        if CFG['safe_cache']:
            store = store and headers['Cache-Control'] == PUBLIC
        if store:
            CACHE[key] = (time.monotonic(), status, dict(headers), body)
        headers['X-Lab-Cache'] = 'MISS' if store else 'BYPASS'
        if store:
            headers['Age'] = '0'
        reply(self, status, headers, body)


if __name__ == '__main__':
    origin = HTTPServer(('127.0.0.1', 8010), Origin)
    edge = HTTPServer(('127.0.0.1', 8080), Edge)
    threading.Thread(target=origin.serve_forever, daemon=True).start()
    print('A10: origin=8010 edge=8080; config=', CFG, flush=True)
    try:
        edge.serve_forever()
    except KeyboardInterrupt:
        pass
    finally:
        edge.server_close()
        origin.shutdown()
        origin.server_close()
```

Ushbu o‘quv server ishlab chiqarish muhiti uchun mo‘ljallanmagan. Faqat loopbackda ishga tushiriladi. Restart barcha eski kesh yozuvlarini yo‘qotadi.

2-ilova. framing.py

~/a10-lab/framing.py fayliga saqlang. Bu dastur tarmoqqa so‘rov yubormaydi; faqat ichidagi tayyor bayt ketma-ketligini tahlil qiladi.

```
# Offline model: no sockets, only supplied byte strings.
probe = b'GET /probe HTTP/1.1\r\nHost: a10.test\r\n\r\n'
body = b'0\r\n\r\n' + probe
head = (b'POST /submit HTTP/1.1\r\nHost: a10.test\r\n'
        + b'Content-Length: ' + str(len(body)).encode() + b'\r\n'
        + b'Transfer-Encoding: chunked\r\n\r\n')
raw = head + body


def boundary(data, mode):
    header, content = data.split(b'\r\n\r\n', 1)
    fields = {}
    for line in header.split(b'\r\n')[1:]:
        name, value = line.split(b':', 1)
        fields.setdefault(name.lower(), []).append(value.strip())
    cl = fields.get(b'content-length', [])
    te = fields.get(b'transfer-encoding', [])
    if mode == 'strict' and (len(cl) > 1 or (cl and te)):
        raise ValueError('REJECT ambiguous framing')
    start = len(header) + 4
    if te and mode != 'CL':
        # This teaching parser accepts only an empty chunked body.
        if te != [b'chunked'] or not content.startswith(b'0\r\n\r\n'):
            raise ValueError('Unsupported teaching sample')
        return start + 5
    return start + (int(cl[0]) if cl else 0)


if __name__ == '__main__':
    print('body_bytes=', len(body))
    for mode in ('CL', 'TE', 'strict'):
        try:
            end = boundary(raw, mode)
            print(mode, 'end=', end, 'remaining=', repr(raw[end:]))
        except ValueError as error:
            print(mode, str(error))
    for label, packet in (
        ('CL-only', head.replace(b'Transfer-Encoding: chunked\r\n', b'')
         + body),
        ('TE-only', head.replace(b'Content-Length: '
         + str(len(body)).encode() + b'\r\n', b'') + body)
    ):
        end = boundary(packet, 'strict')
        print(label, 'accepted; remaining=', repr(packet[end:]))
```